

Lab Exercise 10: Dynamic Storage File Parsing & Word Counter via Collections

Objective: To implement a Java system that reads text files from disk, filters out irregular punctuation marks, and counts words using associative data structures.

Aim: To build a file parsing pipeline using `FileReader` utilities alongside the Java Collections framework.

Theory: Java I/O streams connect physical storage hardware to memory execution pipelines. Using a `HashMap` allows the system to clean and aggregate raw string lines into organized word counts with $O(1)$ lookup and update efficiency.

Algorithm / Procedure:

1. Input: A text file path containing unstructured paragraphs.
2. Instantiate file stream structures inside a robust `try-with-resources` safety block.
3. Read the text line by line, convert words to lowercase, and strip away non-alphabetic punctuation using regex tools (`replaceAll("[^a-zA-Z ]", "")`).
4. Update the word counts dynamically in a `HashMap` using the `.put(word, map.getOrDefault(word, 0) + 1)` method.

Program Code: Problem6_WordFrequencyAnalyzer.java

```
import java.io.*;

import java.util.*;

public class Problem6_WordFrequencyAnalyzer {

    public static Map<String, Integer> calculateFileWordFrequency(String filepath) {

        // TODO: Implement file processing and map aggregation rules here

        return new HashMap<>(); // placeholder

    }

    public static void main(String[] args) {

        // Assume a test file "input.txt" contains: "Hello world! Hello Java."

        Map<String, Integer> frequencyMap = calculateFileWordFrequency("input.txt");

        System.out.println("Word Frequencies: " + frequencyMap);

    }

}
```

```
}  
  
}
```

Sample Input / Output:

Input File Content:

"Java is fun. Learning Java is great!"

Output:

Word Frequencies: {java=2, is=2, fun=1, learning=1, great=1}

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Proper file stream management and exception control (3 Marks)
- Correct use of Collections maps and word cleaning filters (2 Marks)
- Handling missing files or empty text sources safely (2 Marks)
- Proper code indentation and logging comments (1 Mark)